## Introduction

In this report, we will evaluate the impact of New York City's January 2025 congestion pricing implementation on For-Hire Vehicles (FHV). The congestion pricing was introduced by the New York Metropolitan Transportation Authority (MTA). The aim was to mitigate the effects of an $8.3 billion deficit and declining service quality, prompting commuters to drive instead of using public transport, worsening traffic congestion. (Kellison & Hoffer, 2026). We will compare two periods: pre-implementation (January-April 2024) and post-implementation (January-April 2025). The selected periods allow us to meet the size requirement for the processing dataset. Most vehicles are charged once per day going through the toll, whereas taxis and rideshare vehicles are charged per entry at a reduced fare. Toll fees vary by time of day, with full rates applying during peak hours: weekdays from 5 am to 9 pm and weekends from 9 am to 9 pm, with a 75% discount offered during off-peak periods (The MTA, n.d.).

## Processing Objectives:

***What is your central question?***

The central question we seek to answer in processing this large dataset is this:

How did the congestion charge introduced in January 2025 in New York City change Uber and Lyft trip volumes, fares, speeds, and driver pay for trips into and around Manhattan's charging zone?

***Why are you analysing the data?***

New York became the first city in the United States (US) that employed the congestion charge. New York had the worst traffic congestion issue in the country (Kellison & Hoffer, 2026). The Congestion Relief Zone covers Manhattan from 60th Street, heading south. The policy states that the reasons for introducing the charge were not only to reduce the congestion within the central business district, but also to help fund public transportation. We conducted this analysis to assess the fee's effectiveness and its impact on FHV, especially since FHV were tolled on every entry into the zone.

***What are you looking to achieve or obtain from such analysis?***

Over and above the stated central question above, there are a couple of additional sub-questions we seek to answer in our investigation. These include the following:

- We want to measure how trip volumes, average speed, trip length, fare and driver pay have changed for trips that start or end in the charging zone.
- We will use trips that never start or end in the zone as a control group and compare those with trips affected by the policy.

- We want to find when and where the city is the slowest and establish if the charging zone's slowest hours have had an improvement.

## Processing Dataset and Data Preparation

We will be using data from the NYC Taxi and Limousine Commission's (TLC) High Volume For-Hire Vehicle (HVFHV) trip records (New York City Taxi and Limousine Commission, 2026). One Parquet file is produced per month (batch), with one row per Uber or Lyft trip, with 25 columns per file. The TLC started recording trip data for FHV at the beginning of 2015. They did move to a separate dataset for HVFHV's in 2019. For the working dataset that we used in Part 1, we collected data from 27 monthly files (January 2024 – March 2026). This gave us a working dataset with a total size of 13.25GB. We can infer from this that the raw/total dataset must be far larger than this, seeing as we have data all the way from 2015. The processing dataset, which will be used throughout the analysis in this part of the project, consists of 8 months totalling 3.86 GB as downloaded, and 5.19 GB after cleaning and preprocessing.

Due to the ***cbd_merge_fee*** column only coming into existence from January 2025, we had to read the files together with ***mergeSchema***, which allows us to automatically combine varying schemas across multiple files. The missing values from 2024 data were filled in with zeros. We employed some preprocessing steps on the data, which included the following:

- A trip was kept if it was between 0 and 100 miles
- The trip duration was between 0 and 4 hours
- The base fare was between 0$ and $500
- The driver pay was not negative
- The time of the drop-off was after the time of the pickup

These operations eliminated 150 317 rows, leaving us with a total of 159,922,576 rows.

Lastly, we derived 5 new columns to aid in our analysis: ***pickup_day, pickup_hour, pickup_month, trip_minutes*** and ***company*** (Uber or Lyft).

## MapReduce-style Implementation:

The provided Data Dictionary does not mark which zones are inside the charging zone, but the data does provide us with a hint: from 2025, trips that start inside the zone always pay the fee. We ***groupBy*** the ***PULocationID***, which is the pickup zone, and then each group was reduced to its trip count and its number of fee-paying trips. We then proceeded to do a broadcast join with the zone table, which allows Spark to copy the table to every task, instead of shuffling millions of trips. From the notebook results, we can see that 38 zones had between 93-97% of the 2025 trips paying the fee, and the next busiest zone (Governor's Island/Ellis Island/Liberty Island) had 70%

| | | | | | |
|---|---|---|---|---|---|
| **35** | 13 | Manhattan | Battery Park City | 351484 | 0.938 |
| **36** | 12 | Manhattan | Battery Park | 6892 | 0.933 |
| **37** | 88 | Manhattan | Financial District South | 189714 | 0.933 |
| **38** | 105 | Manhattan | Governor's Island/Ellis Island/Liberty Island | 10 | 0.700 |
| **39** | 142 | Manhattan | Lincoln Square East | 524089 | 0.674 |
| **40** | 237 | Manhattan | Upper East Side South | 658017 | 0.635 |

*Figure 1. Image showing the split between charging and non-charging zones*

We proceeded to apply a threshold of 75%, which will allow us to separate these two groups cleanly. We see from this separation that all 38 zones are in Manhattan.

We have some additional mapping where we label every trip's period (whether it was before or after 2025), and for those trips where the trip either started or ended in the 38 zones, we add a ***CBD*** label; otherwise, we label it as ***Outside CBD***.

We used RDDs to map to key-value pairs. This allowed us to have each trip with the key as (***period***, ***area***), and the value would be the numbers we needed to add up, such as ***trip_time***, ***trip_miles***, ***driver_pay*** and so on. Once done, we used reduceByKey, which is now a Spark action and requires the data to actually move. Looking at the RDD lineage graph:

```
reduced = mapped.reduceByKey(add_values)

print(reduced.toDebugString().decode())
```

```
(44) PythonRDD[83] at RDD at PythonRDD.scala:57 []
 |   MapPartitionsRDD[82] at mapPartitions at PythonRDD.scala:169 []
 |   ShuffledRDD[81] at partitionBy at NativeMethodAccessorImpl.java:0 []
 +-(44) PairwiseRDD[80] at reduceByKey at /tmp/ipykernel_2876/907939091.py:14 []
    |   PythonRDD[79] at reduceByKey at /tmp/ipykernel_2876/907939091.py:14 []
    |   MapPartitionsRDD[78] at javaToPython at NativeMethodAccessorImpl.java:0 []
    |   MapPartitionsRDD[77] at javaToPython at NativeMethodAccessorImpl.java:0 []
    |   SQLExecutionRDD[76] at javaToPython at NativeMethodAccessorImpl.java:0 []
    |   MapPartitionsRDD[75] at javaToPython at NativeMethodAccessorImpl.java:0 []
    |   MapPartitionsRDD[74] at javaToPython at NativeMethodAccessorImpl.java:0 []
    |   FileScanRDD[73] at javaToPython at NativeMethodAccessorImpl.java:0 []
```

*Figure 2. RDD Lineage Graph*

We can see the step-by-step history of the transformations that Spark will execute to calculate the final RDD. The number ***(44)*** tells us that there are 44 partitions that the data is split into. ***+-(44) PairwiseRDD*** is where Spark will do the actual execution of ***reduceByKey***. We note here that because Spark will need to aggregate these values across the partitions, a *Shuffle* will need to occur. The final summation will only occur once all intermediate aggregations have been *Shuffled* onto a single partition. We note that the order from the top is: Map -> Shuffle -> Reduce steps.

Finally, we ran ***collect()*** on the data we had run the ***reduceByKey*** function on. For posterity's sake, we added timings so that we could evaluate how long these steps took. ***results = reduced.collect()*** ran for 1859 seconds (31 minutes). Spark relies on lazy evaluation, which means that it queues up data transformations without executing them immediately. Only when an action such as ***collect()*** is called do we then execute the plan. Because we separated the ***reduce*** step from where we do the ***collect()*** by putting them in different cells and timing each, the step where we do the RDD map and reduce only takes 0.4 seconds, which confirms what we understand about lazy evaluation.

We wanted to be able to do a direct comparison between the lower-level Spark RDD API and the higher-level DataFrame API. In the cell that follows the RDD step, we effectively did the same transformation, but this time making use of a DataFrame. For the DataFrame, we used groupBy and aggregated along the period and area as well by using ***trips.groupBy("period", "area").agg***.

The immediate difference we can see is that the DataFrame cell took a total time of 68.67 seconds, which is almost a 30X improvement in processing speed. It produces the exact same output, but because we totalled the RDD per month and the DataFrame per 4-month period, we will need to do a little arithmetic to confirm that the values are the same. For example, the DataFrame has a total of 27983637 trips, and the RDD has 6995909.25 trips per month (***Before 2024*** in the ***CBD area***). If we do 27983637/4, we get 6995909.25 trips.

Returning to the speed difference, we ran the cells multiple times to ensure that it was a reproducible difference every time, and it was. This result is a clear indication that the Catalyst Optimiser used in DataFrames is truly making a difference in how it rewrites the query to be more efficient. Project Tungsten, which manages memory and CPU cycles, also contributes to the runtime difference that we see.

Lastly, on the DataFrame, we discuss the Physical Plan when running ***df_summary.explain(mode="formatted")***:

```
== Physical Plan ==
AdaptiveSparkPlan (6)
+- HashAggregate (5)
   +- Exchange (4)
      +- HashAggregate (3)
         +- Project (2)
            +- Scan parquet  (1)
```

*Figure 3. DataFrame Physical Plan*

The structure used here reflects the Optimiser's choice of writing the query for execution. It also reflects the Map -> Shuffle -> Reduce steps in DataFrame form. *Scan parquet(1)* reads raw data from storage. *Project (2)*, a projection operation that selects only the specific columns needed for the query. *HashAggregate (3)* is a partial aggregation that aggregates partitions locally before sending over the network. *Exchange (4)* would be a *Shuffle* operation. *HashAggregate (5)* is the final aggregation. Each partition now has all the rows for its assigned keys. This is where it will merge the partial results into the final output.

## Results:

*Table 1. Map-reduce Style Results Comparison*

| | period | area | trips_per_month | avg_speed_mph | avg_trip_miles | avg_fare | avg_driver_pay | avg_cbd_fee |
|---|---|---|---|---|---|---|---|---|
| **0** | Before (2024) | CBD | 6995909.25 | 14.46 | 5.40 | 30.63 | 22.21 | 0.00 |
| **2** | After (2025) | CBD | 6654532.25 | 15.15 | 5.51 | 33.00 | 23.00 | 1.44 |
| **1** | Before (2024) | Outside CBD | 13004160.75 | 15.93 | 4.68 | 21.84 | 17.52 | 0.00 |
| **3** | After (2025) | Outside CBD | 13326041.75 | 15.95 | 4.66 | 22.27 | 17.90 | 0.04 |

Looking at *Table 1* above, if we concentrate on the first two rows showing activity within the CBD, we can see that after 2025 the value for trips-per-month lowered; we can see an increase in the average speed – this tells us traffic is moving faster through the CBD. We can see an increase in the trip miles, the average fare, and the average driver fee. Average trip length rising while trip numbers fell suggests the trips that disappeared were mostly short ones. If we contrast this with the activity outside the CBD, we can see an increase in the trips per month in 2025, and the other measures don't have much difference between them. This acts as a control group that shows us that the congestion fee did indeed have a change in the traffic congestion patterns.

The following three figures that we will be referring to can all be found in the Appendices section. *Figure 4* shows us that the slowest traffic movement is between 16:00 and 18:00. Both 2024 and 2025 show the same general trend; however, the 2025 trendline does always hover above the 2024 speeds. This is another indication that the congestion fee has had a positive impact in the charging zone. *Figure 5* confirms what we have seen from *Table 1*, in that outside of the charging zone, there isn't much difference in congestion between 2024 and 2025. We do see that peak traffic shifts a little earlier, though, occurring between 1500 and 1700 hours. Finally, *Figure 6* shows us that daytime traffic for the trips taken per month hasn't had much effect. We do, however, see a difference in the number of trips falling from around 17:00 going into the evening. Evening and late-night trips are more often leisure trips, which are easier to replace or skip than trips to work.

## Business/societal Interpretation:

Our findings show that the congestion charge managed to accomplish the desired outcome, with fewer trips in the charging zone, faster traffic at peak hours and additional funding being generated for public transport (by virtue of the congestion fee itself). For the drivers, they are taking fewer trips into the zone, but the trips are faster on average. This will allow them to complete more trips per hour, as they spend less time sitting in traffic. We saw in *Table 1* that after the introduction of the congestion fee, there was a decrease in trips within the congestion zone, but there was an increase in trips in the rest of the city. This indicates that demand moved from the zone, and the rest of the city is now able to benefit from increased ride-share availability. The evening and late-night drop in trips in the congestion zone matters for businesses in the area such as movie theatres, bars and restaurants. These are the types of businesses that are being impacted (potentially by this change). The people who work late shifts at the previously mentioned businesses could have a safety concern, with there being fewer rides available later in the evenings.

## Conclusion

The 160 million records that we processed answer the questions we had set out to answer at the beginning of this part of the project. We were able to establish that the congestion fee had resulted in a decreased number of trips taken into the congestion zone. We were able to also establish the time that the peak traffic in the zone (and the rest of the city) was, with there being an improvement in the average speed, and therefore a possible increase in the number of trips a driver could take per hour.

## References

Kellison, C., & Hoffer, A. (2026, July 16). *Can New York City Tax Itself Out of Traffic?* Tax Foundation. https://taxfoundation.org/blog/nyc-congestion-pricing-traffic-tax/

New York City Taxi and Limousine Commission. (2026). *TLC Trip Record Data*. New York City Taxi and Limousine Commission. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

The MTA. (n.d.). *About the Congestion Relief Zone toll*. The MTA. Retrieved October 15, 2026, from https://www.mta.info/fares-tolls/tolls/congestion-relief-zone/about

# Appendices:

GitHub Repo Link: https://github.com/molebiemang/mit805-big-data-semester-project

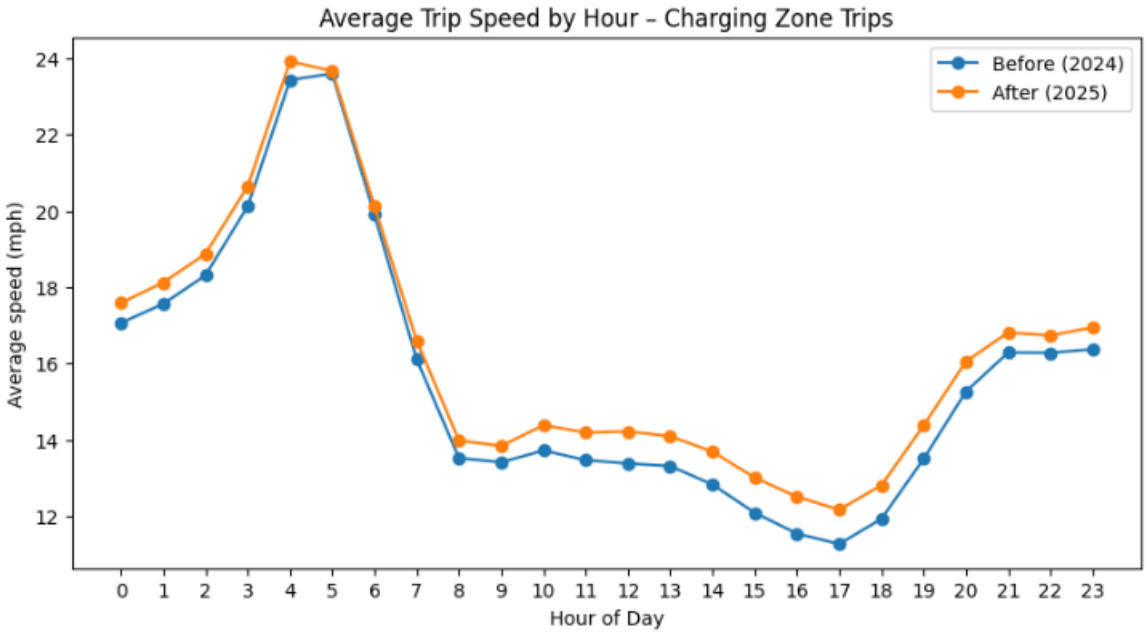



*Figure 4. Average Trip Speed per Hour*

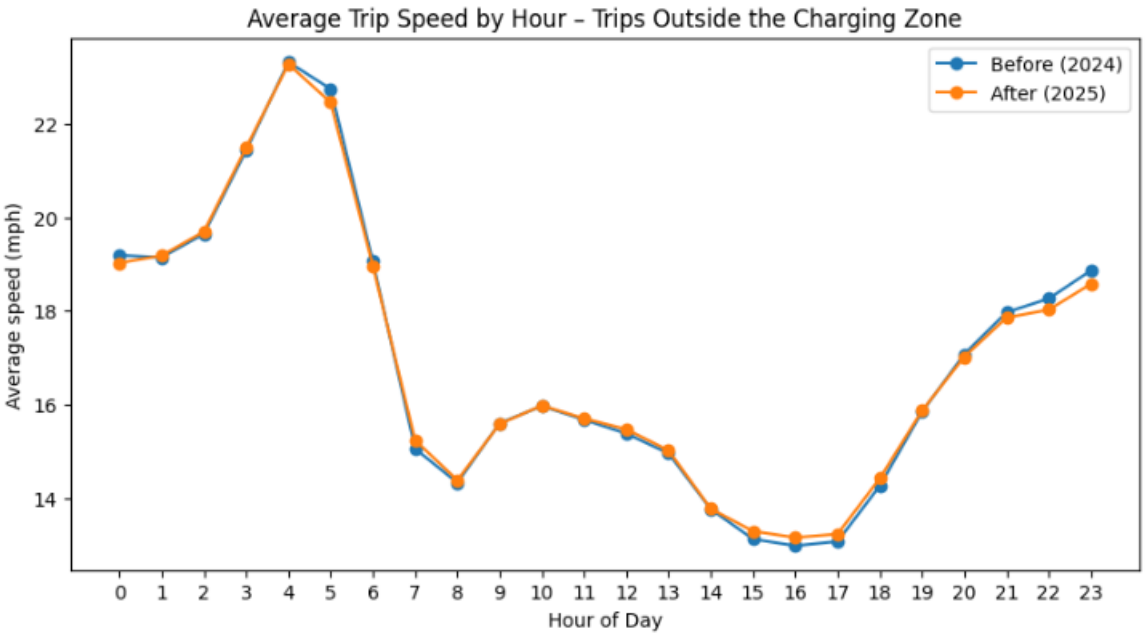



*Figure 5. Average Trip Speed by Hour*

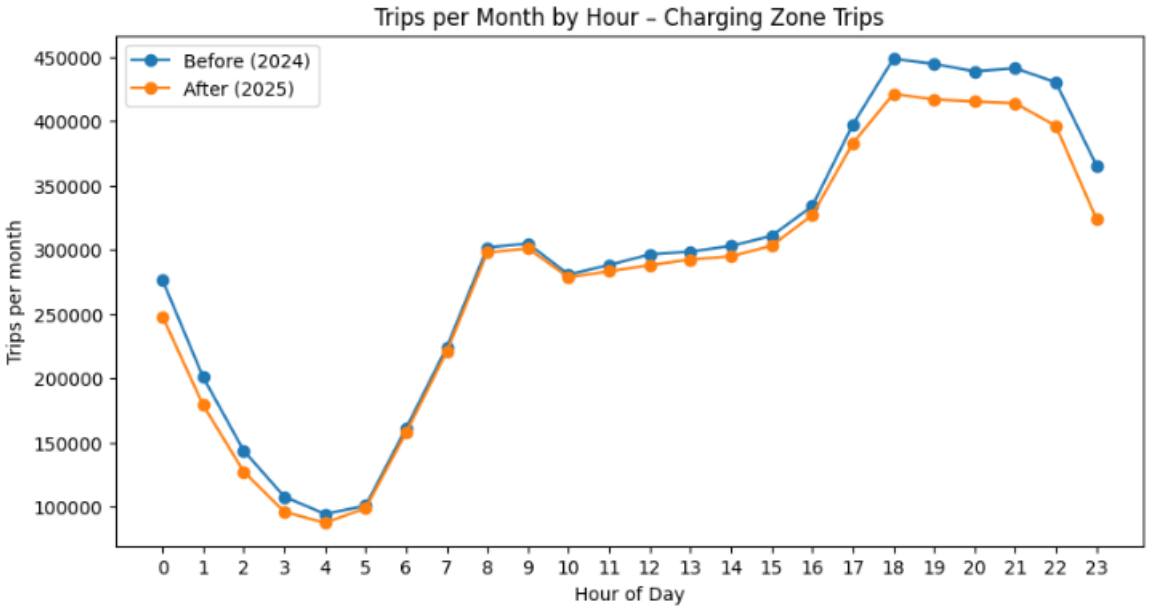



*Figure 6. Trips per Month by Hour*